

Homework 4: RL for Large Language Models

Submission Guidelines: Your deliverables shall consist of 2 separate files – (i) A PDF file: Please compile all your write-ups into one .pdf file (photos/scanned copies are acceptable; please make sure that the electronic files are of good quality and reader-friendly); (ii) A zip file: Please compress all your source code (including `train_dpo.py`) into one .zip file. Please submit your deliverables via E3.

Grading Policy: For HW4, you would receive a basis score of 50 by default, and the resulting HW4 raw score would be $50 + X$, where X denotes the number of total points you earn from the following Problems 1 and 2.

Background Reinforcement Learning from Human Feedback (RLHF) aligns language models with human preferences, but the standard pipeline (*i.e.*, train a reward model, then run PPO) is complex and potentially subject to reward over-optimization. **Direct Preference Optimization (DPO)** (Rafailov et al., 2023) sidesteps explicit reward modeling by showing that the RLHF objective has a closed-form solution expressible as a simple classification loss directly on preference pairs.

Given a preference dataset of triples (x, y_w, y_l) , which consists of a prompt x , a preferred (“chosen”) response y_w , and a rejected response y_l , DPO optimizes:

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right], \quad (1)$$

where β controls the KL penalty strength (*i.e.*, deviation from the reference policy π_{ref}).

Suggested reading before starting:

- Rafailov et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. [arXiv:2305.18290](https://arxiv.org/abs/2305.18290) (especially Sections 1-4).
- DPO Trainer docs in TRL: https://huggingface.co/docs/trl/dpo_trainer

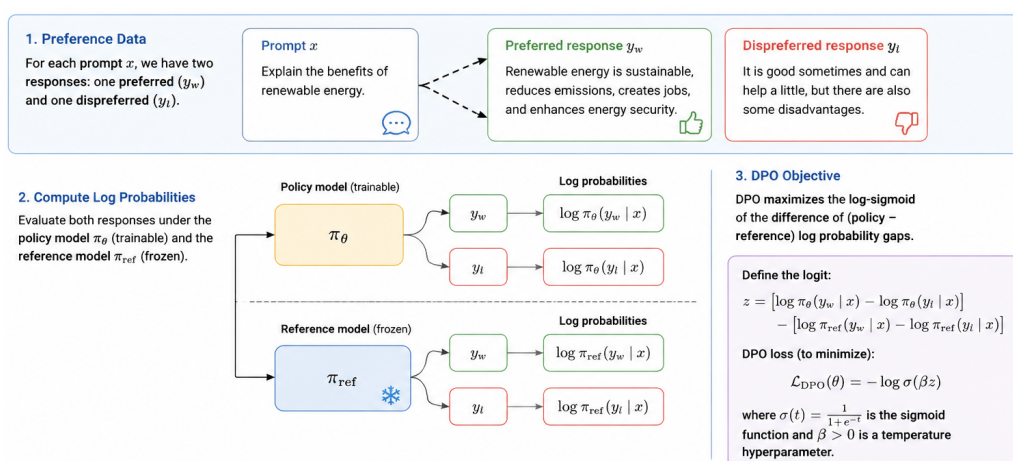


Figure 1: A summary of the DPO framework.

Problem 1 (Principles of DPO and DPO Trainer)

(10+25=35 points)

In this problem, we build on Lecture 21 about DPO and dive deeper to study the principles and the actual implementation of DPO.

(a) Please write down $\nabla_{\theta} \mathcal{L}_{\text{DPO}}$, the (exact) gradient of the DPO loss function in Equation (1). Can you explain the intuition behind this gradient? (Hint: You may refer to Section 4 of the DPO paper)

(b) Let's take a close look at DPOTrainer in the TRL repo (https://github.com/huggingface/trl/blob/main/trl/trainer/dpo_trainer.py) and check the actual implementation. The file is of about 1550 lines. All line numbers refer to the main branch as of June 2026.

- **Question 1:** The DPO loss sums log-probabilities over *response* tokens only, not prompt tokens. Given a batch tensor containing chosen and rejected sequences, explain exactly how the trainer produces a scalar log-probability per sequence? (Hint: `DataCollatorForPreference.torch_call()` at around Line 150 and `_compute_loss()` at around Line 1190)
- **Question 2:** When the “prompt+response” pair exceeds `max_length`, something would get cut or truncated. Is it the beginning or the end of the sequence that survives? Does this mean the prompt or the response gets cut when sequences are too long? (Hint: Check `torch_call()` at around Lines 152-165)
- **Question 3:** In computing the standard DPO loss that involves $\log \sigma(x)$ ($\sigma(\cdot)$ denotes the sigmoid function), the numerical stability is usually an issue. Can you explain how the DPOTrainer handles this? (Hint: Check the function `_compute_loss()` at around Line 1280)
- **Question 4:** There are several subsequent enhancements of DPO loss, such as Implicit Preference Optimization (IPO). Can you explain what IPO actually does in implementation? (Hint: Check Lines 1288-1299)
- **Question 5:** In DPO, a reference policy is needed in computing the token likelihood ratio in the loss function. Can you explain how the reference policy is handled in practice? (Hint: Check `__init__()` at around Lines 570-599 and Lines 762-777)

Problem 2 (DPO on UltraFeedback Benchmark Dataset)

(10+15+10=35 points)

In this problem, you will implement a training script using the provided starter code in `train_dpo_starter.py`. The TODOs in the file tell you exactly what to fill in.

Experimental setup.

- **Model:** Qwen/Qwen2.5-0.5B-Instruct (takes about 1 GB in fp16 and can fit easily within any commercial GPU with 20 GB VRAM even with a reference model copy.)
- **Dataset:** `trl-lib/ultrafeedback_binarized`, a pre-formatted preference dataset ready for DPOTrainer (https://huggingface.co/datasets/trl-lib/ultrafeedback_binarized).

The recommended package versions are as follows:

- **Python:** ≥ 3.9 (tested on 3.10)
- **torch:** ≥ 2.0 (tested on 2.12)
- **transformers:** 5.11
- **trl:** 1.6
- **wandb:** 0.27

(a) (Data Inspection) Before training, let us explore the UltraFeedback dataset:

- Print `dataset[0]` and identify the three keys (`prompt`, `chosen`, `rejected`).
- Compute and report the average token length of `chosen` vs. `rejected` responses using the tokenizer.

Hyperparameter	Value
Model	Qwen/Qwen2.5-0.5B-Instruct
Epochs	1
Batch size (per device)	2
Gradient accumulation steps	4 (effective batch = 8)
Learning rate	5e-7
β	0.1
max_length	1024

(b) (DPO Training) Complete the TODOs in `train_dpo.py` to train the model with the default configuration shown above. Log and report the following evaluation metrics: `rewards/chosen`, `rewards/rejected`, `rewards/margins`, and `loss` over training steps (via W&B plots).

Remark. When `report_to="wandb"` is set, `DPOTrainer` automatically logs the following metrics. Use this as a reference when interpreting your training runs.

Metric	Description	Expected trend
<code>rewards/chosen</code>	Average implicit reward for preferred responses: $\beta \log(\pi_\theta(y_w x) / \pi_{\text{ref}}(y_w x))$	↑ increases
<code>rewards/rejected</code>	Similarly for rejected responses	↓ decreases
<code>rewards/margins</code>	<code>rewards/chosen</code> – <code>rewards/rejected</code> . This is a key summary metric, where a growing margin means the model is learning the preference signal.	↑ increases
<code>rewards/accuracies</code>	Fraction of batch examples where <code>reward(chosen) > reward(rejected)</code> ; preference classification accuracy.	↑ toward 1.0

(c) (Hyperparameter Study) Run the following ablations. Keep all other hyperparameters fixed and vary **one factor at a time**. For each run, record `rewards/margins` at the end of training (or at training step 1000 if the training process takes too long). What interesting findings do you get through these experiments?

Run	β	Learning rate	Notes
A (baseline)	0.1	5e-7	default
B	0.01	5e-7	weak KL penalty
C	0.5	5e-7	strong KL penalty
D	0.1	5e-6	10× higher LR
E	0.1	5e-8	10× lower LR